

FPGA 기반 멀티 모듈 시스템 설계

(Verilog) 초음파 센서, 온도도 센서, 디지털 시계, UART 및 FIFO 연동

AI 시스템반도체설계 2기 8조

정민교 박지훈 임도엽 김민규



대한상공회의소
서울기술교육센터

• 목차

0 1 개요

0 2 초음파 센서 (임도엽)

- | | |
|----------------|-----------------|
| (1) 개요 | (2) 설계 및 구조 |
| (3) 세부 설계 및 검증 | (4) 트러블 슈팅 및 해결 |
| (5) 동작 시연 | (6) 고찰 및 배운 점 |

0 4 UART 및 FIFO 연동

0 3 온/습도 센서 (김민규)

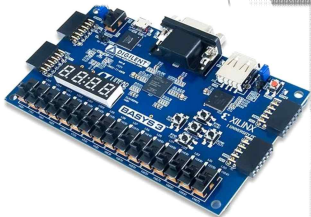
- | | |
|----------------|-----------------|
| (1) 개요 | (2) 설계 및 구조 |
| (3) 세부 설계 및 검증 | (4) 트러블 슈팅 및 해결 |
| (5) 동작 시연 | (6) 고찰 및 배운 점 |

0 5 동작 시연 및 고찰

1. 개요

▶ 프로젝트 목적

- Watch & Stopwatch 기능 구현 | (박지훈)
- 초음파 센서 기능 구현 | (임도엽)
- 온/습도 센서 기능 구현 | (김민규)
- UART 및 FIFO 연동 | (정민교)
- 각 모듈의 simulation 검증
- FPGA 기반 동작 (BASYS3)

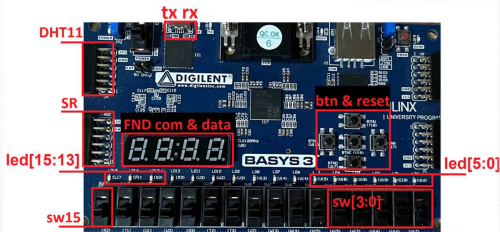


1. 개요

▶ Board Manual

- (1) sw0 : 시간 자릿수 모드
- (2) sw1 : Watch/Stopwatch 모드
- (3) sw2 : 거리 측정 모드
- (4) sw3 : 온/습도 측정 모드
- (5) sw15 : Watch 시간 변경 모드
- (6) led0~3 : Watch/Stopwatch 모드 표시
- (7) led4 : 거리 측정 모드 표시
- (8) led5 : 온/습도 측정 모드 표시
- (9) led12 : 센서 초과 온도 경고등 표시
- (9) led13 : 온/습도 센서 데이터 유효 표시
- (10) led14,15 : Watch 시간 변경 모드 표시

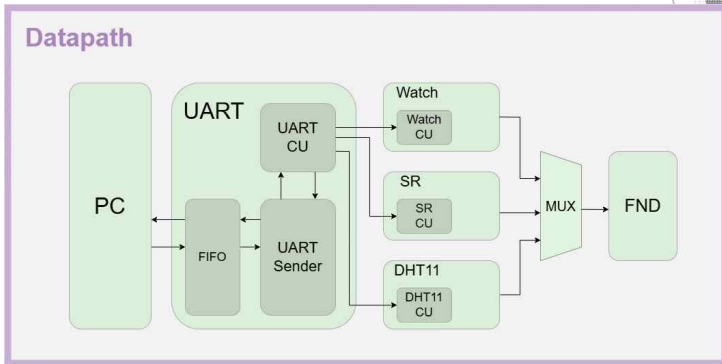
▶ Board Ports



* 목표 : 사용자 친화적인 보드 구현

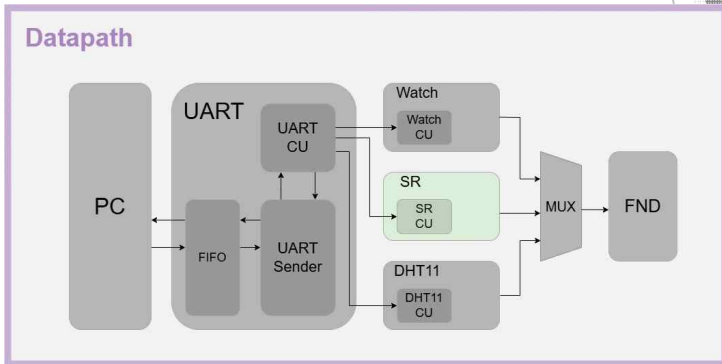
1. 개요

▶ 전체 Datapath



2. 초음파 센서 (임도엽)

▶ Module SR



목차

01 개요

02 설계 및 구조

03 세부 설계 및 검증

04 트러블 슈팅 및 해결

05 고찰 및 배운 점

01 개요

기본 개요

사용 보드

Basys3

개발 언어

Verilog

시뮬레이션

Vivado

구현 대상

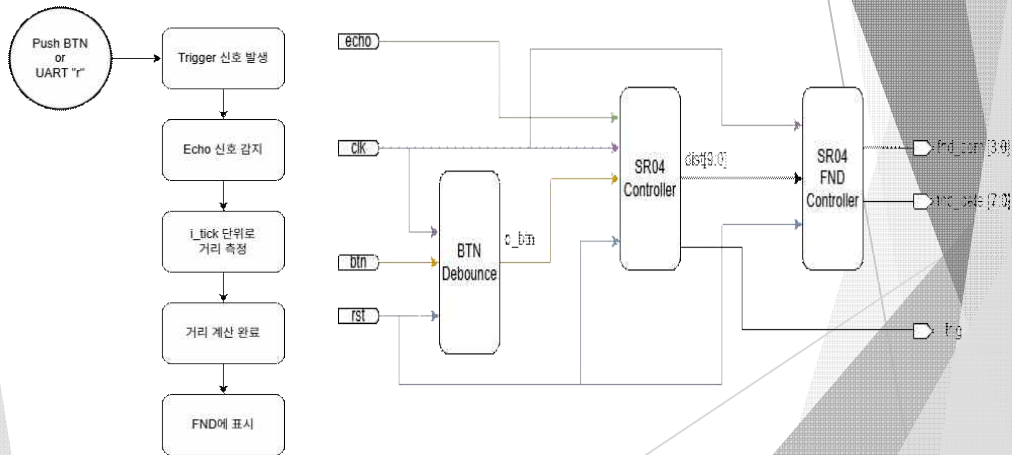
초음파 거리 측정

프로젝트 목표

- 초음파 센서를 통한 거리 측정
- FSM과 Timing에 대한 이해
- 각 모듈의 simulation 검증
- FPGA 기반 동작 (BASYS3)

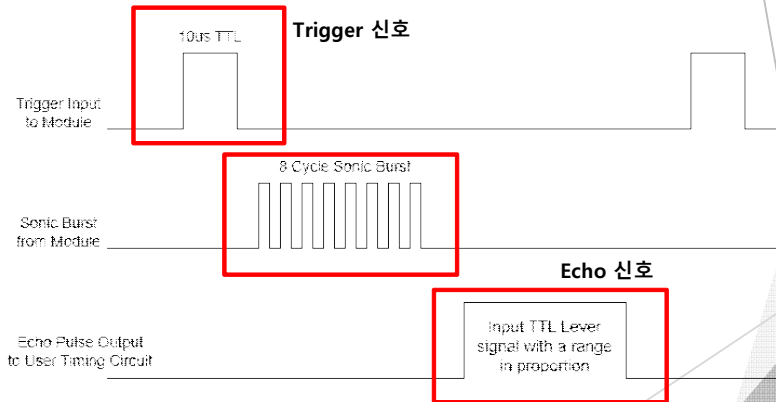
02 설계 및 구조

▶ Top Module Block Diagram & Flow Chart



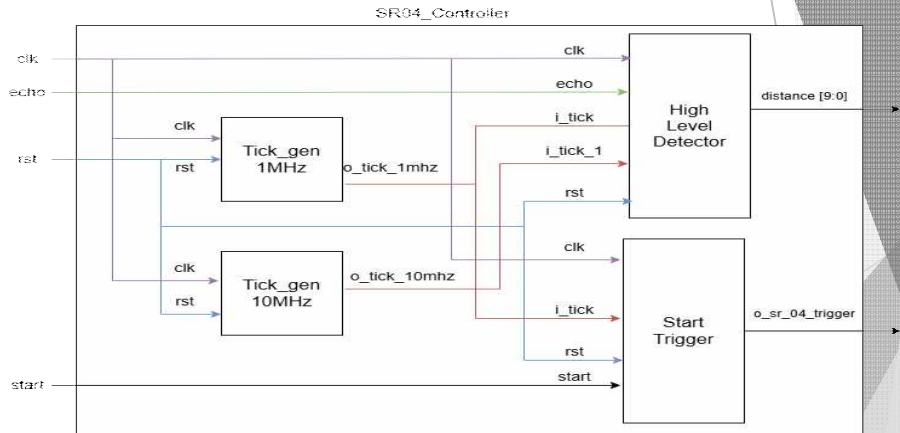
02 설계 및 구조

▶ Timing Diagram



03 세부 설계 및 검증

▶ SR04 Controller – Block Diagram



03 세부 설계 및 검증

▶ Tick_gen_1MHz - Code

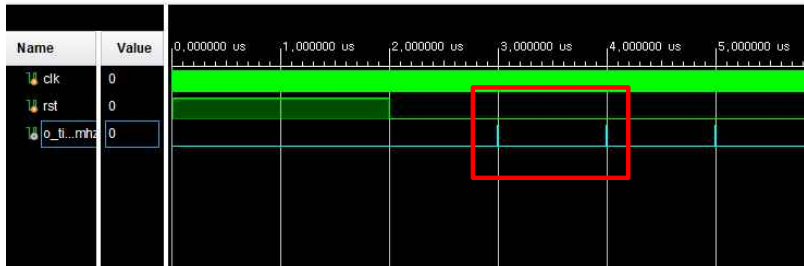
Count가 분주비에 도달하면 tick = 1
이후 count 리셋

Tick 신호는 1us 주기로 발생, 펄스 폭 10ns
=> 거리 측정을 위한 타이밍 기준 신호

```
module tick_gen_1MZ (  
    input  clk,  
    input  rst,  
    output o_tick_1mhz  
);  
    parameter F_COUNT = (100 - 1);  
  
    reg [6:0] count;  
    reg tick;  
  
    assign o_tick_1mhz = tick;  
  
    always @(posedge clk, posedge rst) begin  
        if (rst) begin  
            count <= 0;  
            tick <= 0;  
        end else begin  
            if (count == F_COUNT) begin  
                count <= 0;  
                tick <= 1'b1;  
            end else begin  
                count <= count + 1;  
                tick <= 1'b0;  
            end  
        end  
    end  
end  
endmodule
```


03 세부 설계 및 검증

▶ Tick_gen_1MHz - Simulation



03 세부 설계 및 검증

▶ Tick_gen_10MHz - Code

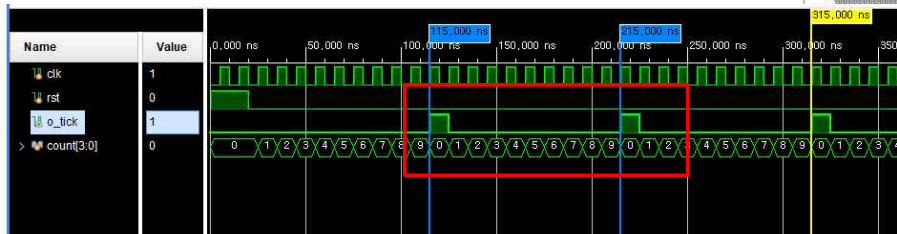
Count가 분주비에 도달하면 tick = 1
이후 count 리셋

Tick 신호 1us 주기로 발생, 펄스 폭 100ns
=> 마이크로초 단위 시간 측정을 위한 기준
신호

```
module tick_gen_10Mhz (  
    input  clk,  
    input  rst,  
    output o_tick_100khz  
);  
  
    parameter FCOUNT = 10 - 1;  
  
    reg [3:0] count;  
    reg tick;  
  
    assign o_tick_100khz = tick;  
  
    always @(posedge clk, posedge rst) begin  
        if (rst) begin  
            count <= 0;  
            tick  <= 0;  
        end else begin  
            if (count == FCOUNT) begin  
                count <= 0;  
                tick  <= 1'b1;  
            end else begin  
                count <= count + 1;  
                tick  <= 1'b0;  
            end  
        end  
    end  
endmodule
```

03 세부 설계 및 검증

▶ Tick_gen_10MHz - Simulation



03 세부 설계 및 검증

▶ Start Trigger - Code

상태 0

- Start 신호가 들어오면 상태 1로 전이

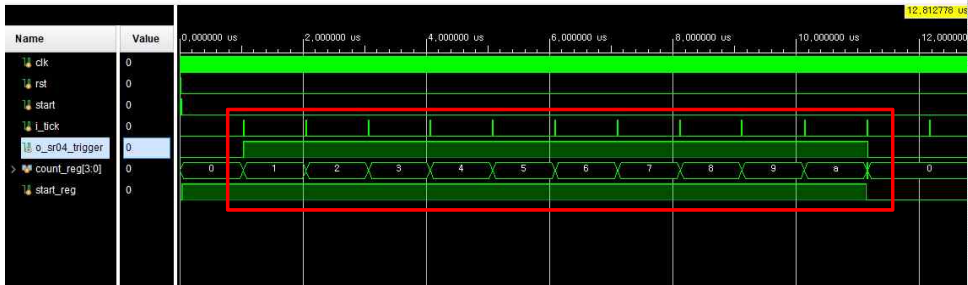
상태 1

- 1us마다 tick 카운트
- 10us 되면 상태 0으로 복귀
- 트리거 출력 유지

```
always @(*) begin
    start_next = start_reg;
    sr04_trigger_next = sr04_trigger_reg;
    count_next = count_reg;
    case (start_reg)
        0: begin
            count_next = 0;
            sr04_trigger_next = 1'b0;
            if (start) begin
                start_next = 1;
            end
        end
        1: begin
            if (i_tick) begin
                sr04_trigger_next = 1'b1;
                if (count_reg == 10) begin
                    start_next = 0;
                end
                count_next = count_reg + 1;
            end
        end
    endcase
end
endmodule
```

03 세부 설계 및 검증

▶ Start Trigger - Simulation



트리거 출력이 1us 간격 tick 기준 10번만 유지

03 세부 설계 및 검증

▶ Start Trigger - Simulation



동작 중 Start신호가 다시 들어와도 무시

03 세부 설계 및 검증

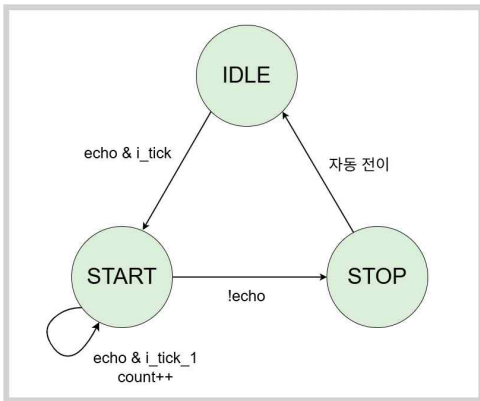
▶ High Level Detector - Code

- 측정된 tick 수를 count_reg에 저장
- assign dist = count_reg / 58 을 통해 cm단위로 변환하여 출력

```
module high_level_detector(  
    input echo,  
    input i_tick,  
    input i_tick_1,  
    input clk,  
    input rst,  
    output [10:0] dist,  
    output dist_done  
);  
  
parameter idle = 0, start = 1, stop = 2;  
  
reg [1:0] c_state, n_state;  
reg [20:0] count_reg, count_next;  
reg dist_reg, dist_next;  
reg r_done, n_done;  
  
assign dist_done = dist_reg;  
assign dist = count_reg / 58;  
  
always @(posedge clk or posedge rst) begin  
    if (rst) begin  
        c_state <= idle;  
        count_reg <= 0;  
        dist_reg <= 0;  
    end else begin  
        c_state <= n_state;  
        count_reg <= count_next;  
        dist_reg <= dist_next;  
    end  
end  
end
```

03 세부 설계 및 검증

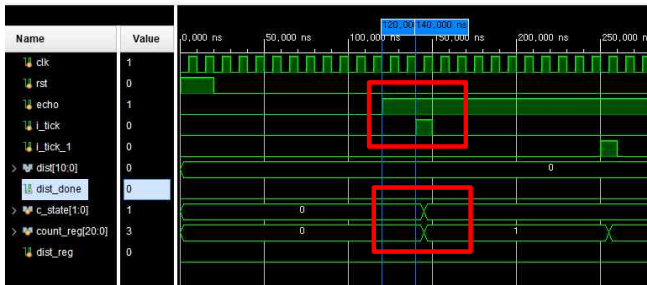
▶ High Level Detector – FSM & Code



```
always @(*) begin
    n_state = c_state;
    count_next = count_reg;
    n_done = r_done;
    dist_next = dist_reg;
    case(c_state)
        idle: begin
            dist_next = 0;
            if (echo & i_tick) begin
                count_next = 1;
                n_state = start;
            end else begin
                count_next = count_reg;
            end
        end
        start: begin
            if (echo & i_tick_1) begin
                count_next = count_reg + 1;
            end else if (!echo) begin
                n_state = stop;
            end
        end
        stop: begin
            n_state = idle;
            dist_next = 1;
        end
    endcase
end
```


03 세부 설계 및 검증

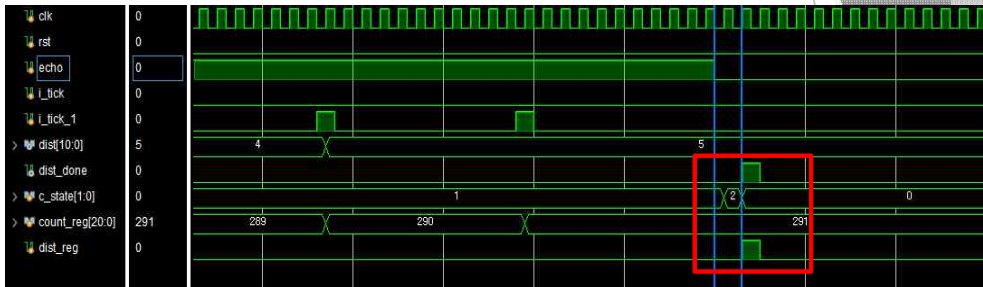
▶ High Level Detector - Simulation



echo신호와 i_tick 두 신호가 들어와야
state -> START로 전이

03 세부 설계 및 검증

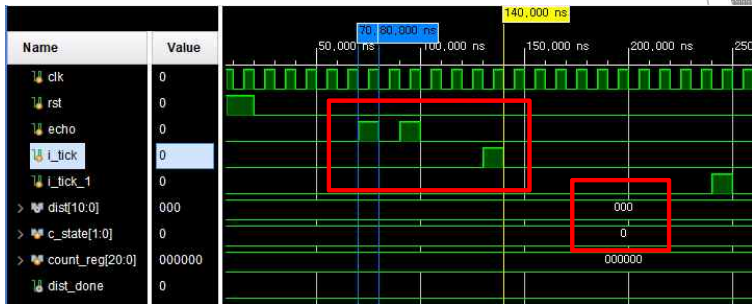
▶ High Level Detector - Simulation



echo신호 low 상태, STOP으로 전이된 후
Dist_done = 1

03 세부 설계 및 검증

▶ High Level Detector - Simulation



Echo가 너무 짧거나 i_tick과의 타이밍이 맞지 않으면 상태 전이와 거리 측정 X

03 세부 설계 및 검증

▶ Uart 연결

```
sender_uart U_SEND (  
    .clk(clk),  
    .rst(rst),  
    .rx(rx),  
    .i_send_data({w_distance, w_rh_data, w_t_data}),  
    .dht_done(w_dht_done),  
    .dht_valid(dht11_valid),  
    .dist_done(w_dist_done),  
    .rx_done(rx_done),  
    .rx_data(rx_data),  
    .tx(tx)  
    // .tx_done(tx_done)  
);
```

```
else if (r_dist_done) begin  
    if (send_cnt_reg < 17) begin  
        // 상위부터 보내기  
        case (send_cnt_reg)  
            5'h00: send_data_next = 8'h44; // D  
            5'h01: send_data_next = 8'h49; // I  
            5'h02: send_data_next = 8'h53; // S  
            5'h03: send_data_next = 8'h54; // T  
            5'h04: send_data_next = 8'h41; // A  
            5'h05: send_data_next = 8'h4E; // N  
            5'h06: send_data_next = 8'h43; // C  
            5'h07: send_data_next = 8'h45; // E  
            5'h08: send_data_next = 8'h3A; // :  
            5'h09: send_data_next = w_dist_send_data[31:24]; // dist_data  
            5'h0a: send_data_next = w_dist_send_data[23:16]; // dist_data  
            5'h0b: send_data_next = w_dist_send_data[15:8]; // dist_data  
            5'h0c: send_data_next = 8'h2E; // .  
            5'h0d: send_data_next = w_dist_send_data[7:0]; // dist_data  
            5'h0e: send_data_next = 8'h63; // c  
            5'h0f: send_data_next = 8'h6D; // m  
            5'h10: send_data_next = 8'h0A; // \n  
        endcase  
        send_cnt_next = send_cnt_reg + 1;  
    end else begin  
        n_state = 0;  
        send_next = 0;  
    end  
end
```

04 트러블 슈팅 및 해결

▶ Timing Issue

```
always @(*) begin
    state_next = state_reg;
    count_next = count_reg;
    distance_next = distance_reg;
    dist_done_next = dist_done_reg;
    case (state_reg)
        0: begin
            if (echo) begin
                state_next = 1;
                count_next = 0;
            end else begin
                count_next = count_reg;
            end
        end
        1: begin
            if (echo && i_tick) begin
                count_next = count_reg + 1;
            end else if (echo == 0) begin
                distance_next = count_reg / 58;
                dist_done_next = 1;
                state_next = 0;
            end
        end
    endcase
end
endmodule
```



```
assign dist = count_reg / 58;

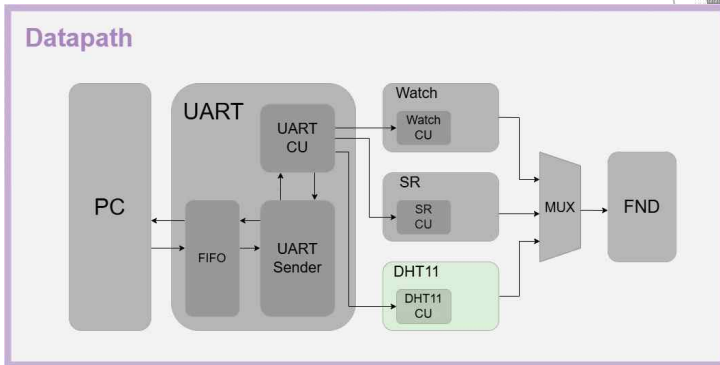
always @(*) begin
    n_state = c_state;
    count_next = count_reg;
    n_done = r_done;
    dist_next = dist_reg;
    case(c_state)
        idle: begin
            dist_next = 0;
            if (echo & i_tick) begin
                count_next = 1;
                n_state = start;
            end else begin
                count_next = count_reg;
            end
        end
        start: begin
            if (echo & i_tick_1) begin
                count_next = count_reg + 1;
            end else if (!echo) begin
                n_state = stop;
            end
        end
        stop: begin
            n_state = idle;
            dist_next = 1;
        end
    endcase
end
```

05 고찰 및 배운 점

- (1) RTL schematic -> simulation을 확인해야 빠른 디버깅이 가능하다는 것을 체감
- (2) 신호의 타이밍 제어와 동기 회로의 중요성 체감
- (3) 단순 구현이 아닌 정확성과 안정성을 고려한 설계의 중요성 습득
- (4) 시뮬레이션 파형 분석을 통한 동작 이해도 향상

3. 온/습도 센서 (김민규)

▶ Module DHT11



목차

1. 목적

2. dht11_data_sheet

3. communication process

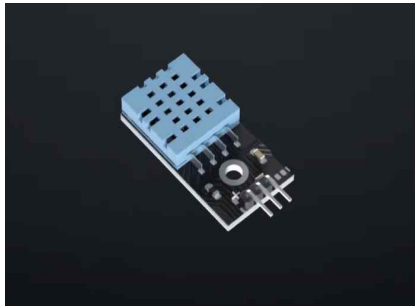
4. 설계 구조

5. 추가 기능

6. 트러블 슈팅

7. 고찰

1. 목적



- 온습도 측정 기능 구현
- 모듈의 simulation 검증
- FPGA 기반 동작 학습
- 추가 기능 구현

2. Dht11_data_sheet

Parameters	Conditions	Minimum	Typical	Maximum
Humidity				
Resolution		1%RH	1%RH	1%RH
			8 Bit	
Repeatability			± 1%RH	
Accuracy	25°C		± 4%RH	
	0-50°C			± 5%RH
Interchangeability	Fully Interchangeable			
Measurement Range	0°C	30%RH		90%RH
	25°C	20%RH		90%RH
	50°C	20%RH		80%RH
Response Time (Seconds)	1/e(63%)25°C, 1m/s Air	6 S	10 S	15 S
Hysteresis			± 1%RH	
Long-Term Stability	Typical		± 1%RH/year	
Temperature				
Resolution		1°C	1°C	1°C
		8 Bit	8 Bit	8 Bit
Repeatability			± 1°C	
Accuracy		± 1°C		± 2°C
Measurement Range		0°C		50°C
Response Time (Seconds)	1/e(63%)	6 S		30 S

◆ 센서 기능

- 온도 및 습도 측정
- 1-wire 출력

◆ 측정 범위 및 정밀도

- 습도: 20~90%RH ±5%
- 온도: 0~50°C ±2°C
- 응답 시간: 6~15초 (습도), 6~30초 (온도)

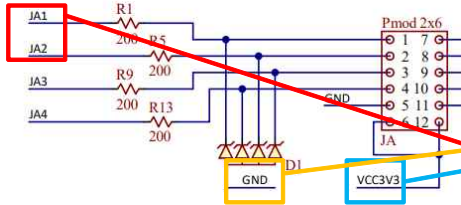
◆ 전기적 특성

- 전원: 3V~5.5V DC
- 평균 소비 전류: 0.2~1mA
- 통신: 단일 버스 (Single-wire) 양방향 통신
- 전송 데이터: 40bit
(습도 16bit (정수, 실수)
+ 온도 16bit (정수, 실수)
+ Checksum 8bit)

Concullusion detect X!!



► board in & out



3. communication process

5.1 Overall Communication Process (Figure 2, below)

When MCU sends a start signal, DHT11 changes from the low-power-consumption mode to the running-mode, waiting for MCU completing the start signal. Once it is completed, DHT11 sends a response signal of 40-bit data that include the relative humidity and temperature information to MCU. Users can choose to collect (read) some data. Without the start signal from MCU, DHT11 will not give the response signal to MCU. Once data is collected, DHT11 will change to the low-power-consumption mode until it receives a start signal from MCU again.

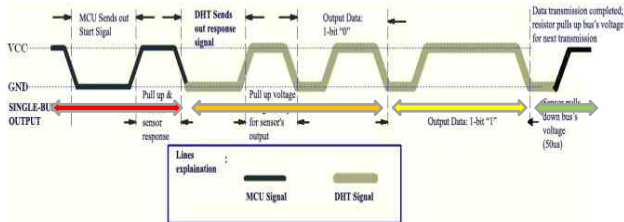
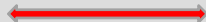


Figure 2 Overall Communication Process

4가지 section

1. START (FPGA->dht11)



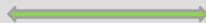
2. RESPONSE (dht11-> FPGA)



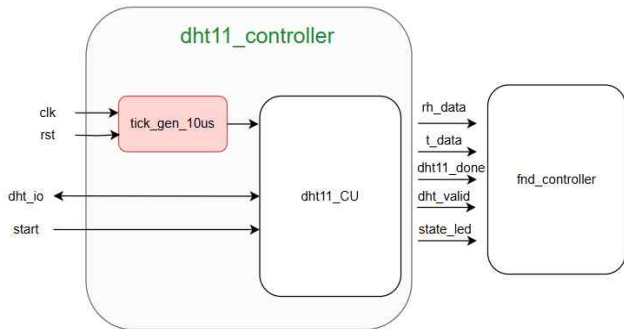
3. GET_DATA (dht11->FPGA)



4. STOP (dht11->FPGA)

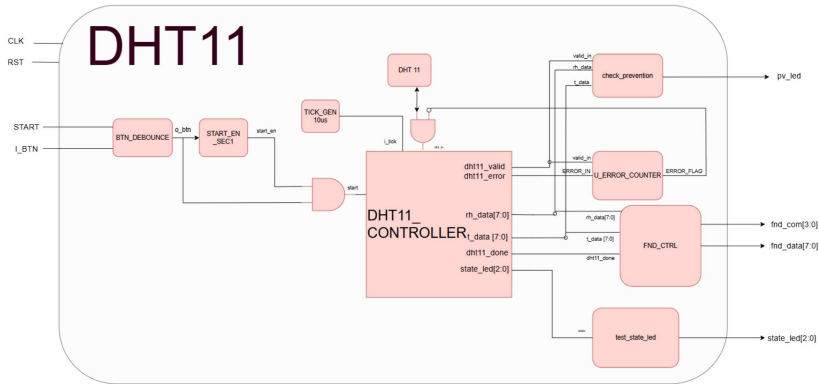


4. 설계 구조

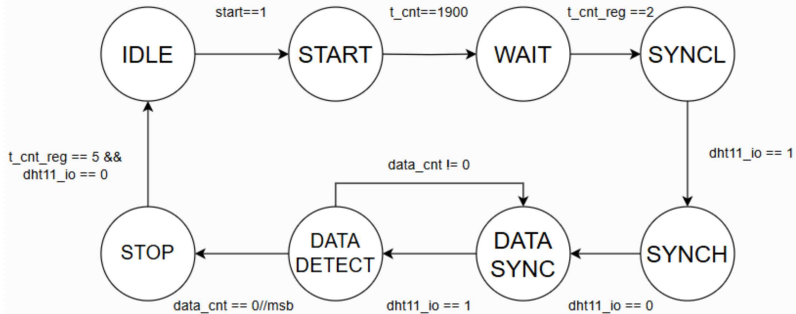


- `tick_gen_10us`를 기준으로 `dht_io`를 감시하여 정해진 process를 수행한다.

4. 설계 구조



FSM



▶ 자세한 내용은 코드와 함께 설명

Fsm CL logic with code

```

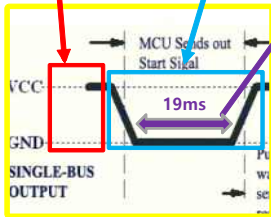
IDLE: begin
    dht11_next = 1;
    io_en_next = 1;
    data_cnt_next = 40;
    valid_next = 0;
    if (start) begin
        n_state = START;
    end
end

```

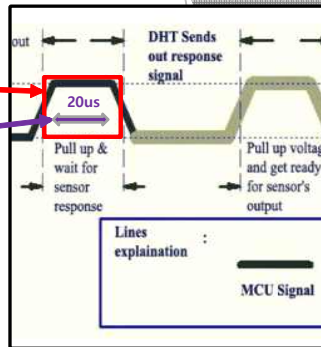
```

START: begin
    if (w_tick) begin
        dht11_next = 0;
        if (t_cnt_reg == 1900 - 1) begin
            n_state = WAIT;
            t_cnt_next = 0;
        end else begin
            t_cnt_next = t_cnt_reg + 1;
        end
    end
end

```




```
WAIT: begin
    // 출력 HIGH
    dht11_next = 1;
    if (w_tick) begin
        if (t_cnt_reg == 2) begin
            n_state = SYNCL;
            t_cnt_next = 0;
            // 출력을 입력으로 전환
            io_en_next = 0;
        end else begin
            t_cnt_next = t_cnt_reg + 1;
        end
    end
end
```



```

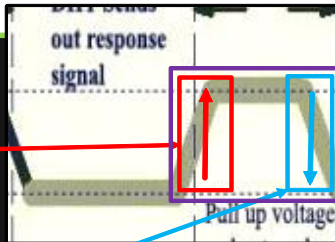
SYNCL: begin
  if (w_tick) begin
    if (dht11_io) begin
      n_state = SYNCH;
    end
  end
end
end

```

```

SYNCH: begin
  if (w_tick) begin
    if (!dht11_io) begin
      n_state = DATA_SYNC; // 정상적으로 LOW 감지
      t_cnt_next = 0;
    end else if (t_cnt_reg >= 400) begin // 타임아웃 설정 (예: 400 * 10us = 4ms)
      n_state = IDLE; // 타임아웃 시 초기화
      t_cnt_next = 0;
    end else begin
      t_cnt_next = t_cnt_reg + 1;
    end
  end
end
end

```



io의 상승을 w_tick이 감지하면 DETECT로 넘김

DATA_SYNC: begin

```
if (data_cnt_reg == 0) begin
    t_cnt_next = 0;
    n_state = STOP;
end else if (w_tick) begin
    if (dht11_io) begin
        n_state = DATA_DETECT;
    end
end
end
```

DATA_DETECT: begin

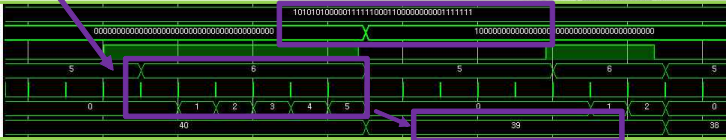
```
if (w_tick) begin
    if (dht11_io) begin
        t_cnt_next = t_cnt_reg + 1;
    end else if (!dht11_io) begin
        if (t_cnt_reg > 4) begin
            data_next[data_cnt_reg-1] = 1'b1;
            t_cnt_next = 0;
        end else begin
            data_next[data_cnt_reg-1] = 1'b0;
            t_cnt_next = 0;
        end
        data_cnt_next = data_cnt_reg - 1;
        n_state = DATA_SYNC;
    end
end
end
```



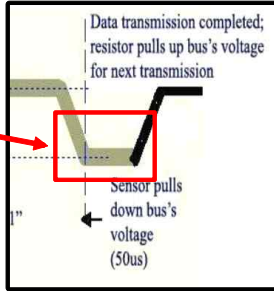
40us보다 짧거나 긴 것 으로 0과 1을 판별

```
> dht11_test_data[39:0]
> data_reg[39:0]
dht11_io
> state_led[2:0]
w_tick
> t_cnt_reg[10:0]
> data_cnt_reg[5:0]
```

```
101010100000111111
10000000000000000000
0
5
0
0
38
```



```
STOP: begin
  if (w_tick) begin
    if (t_cnt_reg == 5) begin
      t_cnt_next = 0;
      n_state = IDLE;
    end else begin
      t_cnt_next = t_cnt_reg + 1;
    end
  end
end
```



DATA_DETECT CORRECT!!

> dht11_test_data[39:0]	10101010000	1010101000001111110001100000000001111111
> data_reg[39:0]	10101010000	1010101000001111110001100000000001111111
> state_led[2:0]	0	7 0
> data_cnt_reg[5:0]	40	0 40
> t_cnt_reg[10:0]	000	003 004 005 000

Circuit Logic test

Case 1. 정상 동작

Case 2. Start 신호 길이 신뢰성 검사

Case 3. Start 신호는 정상이나 DHT11 응답 없음

Case 4. 응답은 정상인데 데이터 전송 중 멈춤

Case 5. Checksum 오류

Case 6. 타이밍 경계 값 조건 검사(데이터 0과 1 경계)

Case 7. 데이터 비트 수 부족

Case 8. 데이터 비트 수 과다

Case 9. dht11_io 핀 노이즈

Case 10. MCU start 신호 연속 반복

Case 11. 전원 불안정 (Reset 도중 Start)

추가 기능 (Error counter)

```
module error_counter #(
    parameter ERROR_THRESHOLD = 4
) (
    input clk,
    input rst,
    input error_in,
    input valid_in,
    output reg error_flag,
    output reg [$clog2(ERROR_THRESHOLD)-1:0] error_count
);
```

FPGA 출력 상태에서 센서가 반대 신호로 라인을 끌어내리면 과전류로 인해 IO 핀 손상 및 수명 단축 위험이 큼니다.

```
assign dht11_io = (io_en_reg & (!error_flag)) ? dht11_reg : 1'bz;
```

```
always @(*) begin
    error_count_next = error_count;
    error_flag_next = error_flag;

    if (valid_in) begin
        // 정상 데이터 수신 시 에러 카운터 리셋
        error_count_next = 0;
        error_flag_next = 0;
    end else if (error_in) begin
        // 에러 발생 시 카운터 증가
        if (error_count < ERROR_THRESHOLD) begin
            if (error_count == ERROR_THRESHOLD)
                error_count_next = error_count + 1;
            end
            if (error_count + 1 >= ERROR_THRESHOLD) begin
                error_flag_next = 1;
            end
        end
    end
end
endmodule
```

추가 기능 (start counter)

6. Electrical Characteristics

VDD=5V, T = 25°C (unless otherwise stated)

	Conditions	Minimum	Typical	Maximum
Power Supply	DC	3V	5V	5.5V
Current Supply	Measuring	0.5mA		2.5mA
	Average	0.2mA		1mA
	Standby	100uA		150uA
Sampling period	Second	1		

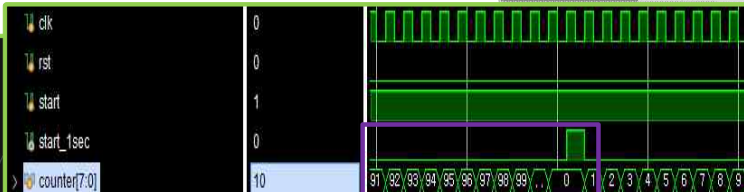
Note: Sampling period at intervals should be no less than 1 second.

DHT11 sensor in the non-normal working conditions will accelerate sensor's aging process.

dht 11 datasheet p8 7. attention 일부 발췌

```
//SL
always @(posedge clk or posedge rst) begin
    if (rst) begin
        c_state <= IDLE;
        start_1sec <= 0;
        counter <= 0;
    end else begin
        c_state <= n_state;
        start_1sec <= (c_state == PROCESS);
        if (c_state == WAIT_1SEC)
            counter <= counter + 1;
        else
            counter <= 0;
    end
end
```

```
// CL
always @(*) begin
    n_state = c_state;
    case (c_state)
        IDLE: begin
            if (start)
                n_state = PROCESS;
            end
        PROCESS: begin
            n_state = WAIT_1SEC;
            end
        WAIT_1SEC: begin
            if (counter >= 100 - 1) // 100_000_000
                n_state = IDLE;
            end
    endcase
end
endmodule
```



CL LOGIC:

process일 때 한 클럭만 받는다
count를 다 세기 전까진 안받는다
1초 이후 IDLE에서 start 받을 준비.

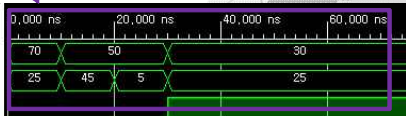
추가 기능 (check_prevention)

```
module check_prevention (  
    input clk,  
    input rst,  
    input [7:0] rh_data,  
    input [7:0] t_data,  
    output reg prevention_led  
);
```

```
always@(posedge clk, posedge rst)begin  
    if(rst)begin  
        prevention_led <= 0;  
    end else begin  
        if(rh_data > 8'd60 || (t_data > 8'd40 || t_data < 8'd10))begin  
            prevention_led = 0;  
        end else begin  
            prevention_led = 1;  
        end  
    end  
end  
end  
endmodule
```

• Storage:
Keep the sensor at temperature 10-40°C,
humidity <60%RH.

Name	Value
> rh_data[7:0]	30
> t_data[7:0]	25
prevention_led	1



3. 트러블 슈팅

(1) SYNC 과정에서 자꾸 멈추는 문제

버튼을 연속으로 누르면 가끔씩 멈추는 문제가 존재했습니다.

state_led를 통해 금방 문제를 확인.

할 수 있었고, 일정 시간 이상 딜레이 된다면 초기 상태로 돌아가도록 설계했습니다.

```
SYNC: begin
    if (w_tick) begin
        if (dht11_io) begin
            n_state = SYNC;
        end
    end
end

SYNC: begin
    if (w_tick) begin
        if (!dht11_io) begin
            n_state = DATA_SYNC; // 정상적으로 LOW 감지
            t_cnt_next = 0;
        end else if (t_cnt_reg >= 400) begin // 타임아웃 설정 (예: 400 * 10us = 4ms)
            n_state = IDLE; // 타임아웃 시 초기화
            t_cnt_next = 0;
        end else begin
            t_cnt_next = t_cnt_reg + 1;
        end
    end
end
```

Single-bus data format is used for communication and synchronization between MCU and DHT11 sensor. One communication process is about 4ms.

3. 트러블 슈팅

(2) 가끔씩 이상한 값이 출력되는 문제

sampling 주기를 지키기 위한 start_en 신호를 만들었다.

이로 인해 샘플링 주기는 start를 누른 이후 1초로 지켜지게 되었다.

6. Electrical Characteristics

VDD=5V, T = 25°C (unless otherwise stated)

	Conditions	Minimum	Typical	Maximum
Power Supply	DC	3V	5V	5.5V
Current Supply	Measuring	0.5mA		2.5mA
	Average	0.2mA		1mA
	Standby	100uA		150uA
Sampling period	Second	1		

Note: Sampling period at intervals should be no less than 1 second.

DHT11 sensor in the non-normal working condions will accelerate sensor's aging process.

출처: dht11 데이터 시트 p9.

3. 트러블 슈팅

(3) 센서의 고장 대처

데이터 시트의 해결방안을 참고 해서

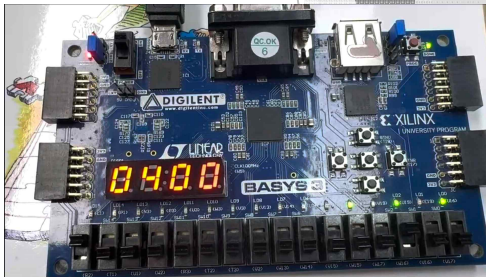
DOWN_COUNTER 설계

센서 수량이 제한적이라면 해결 방안을 시도해 볼 수 있다.

Step one: Keep the DHT sensor at the condition of Temperature 50~60Celsius, humidity <10%RH for 2 hour

Step two: keep the DHT sensor at the condition of Temperature 20~30Celsius, humidity >70%RH for 5 hours.

출처: dht11 데이터 시트 p9.



4. 고찰

(1) 협업 경험의 문제점과 개선점

모듈을 합칠 때 양식이 다르고, 진행도를 기록하지 않아 미완성된 코드를 전달해 누락된 문제가 있었다. 미리 팀원끼리 모듈 통합 시 코딩 규칙을 정하고 공유하는 것이 중요하다고 느꼈다.

(2) DHT11 센서의 타이밍 문제와 배운 점

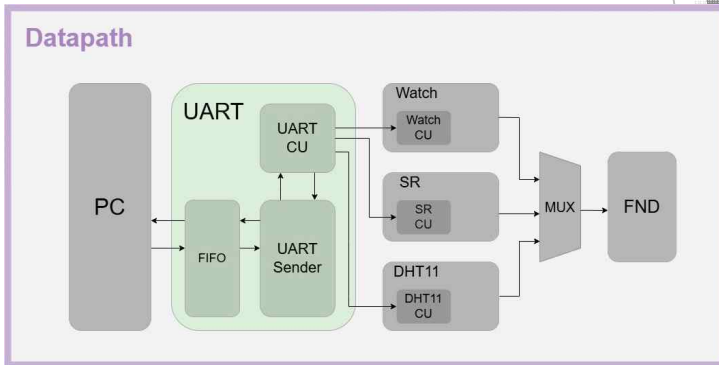
DHT11 센서는 전압 높낮이(하이/로우)로 신호를 측정하는데, 타이밍에 매우 민감했다. 느낀 점: 타이밍이 중요한 센서는 신호 측정을 위한 정확한 코드 작성과 정밀한 디버깅의 필요성을 느꼈다.

(3) 데이터 시트 미숙지 문제와 배운 점

코드 작성 전에 데이터 시트를 꼼꼼히 확인하고, 시간 단위의 동작 방식을 이해하는 과정이 필수적이라는 점을 배웠다.

4. UART 및 FIFO 연동

▶ Module UART



4. UART 및 FIFO 연동

▶ FIFO 구조

ex) UART Tx - FIFO



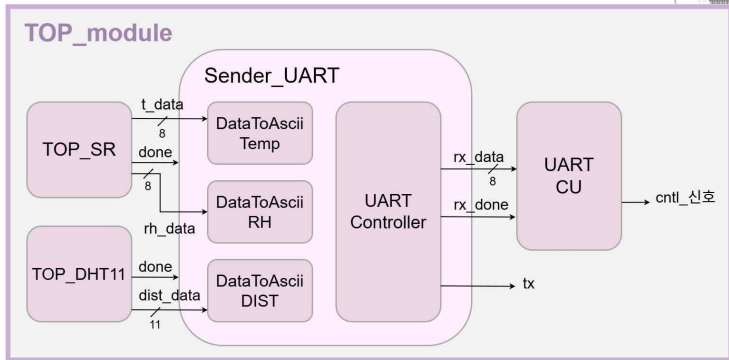
- (1) 센서 : 한 번에 많은 데이터 생성
- (2) FIFO : 데이터 임시 저장
- (3) UART : FIFO에서 하나씩 꺼내 전송

▶ FIFO 기능

- (1) 속도 보완
- (2) 데이터 처리가 불가능할 때,
버퍼에 저장되면 데이터 유실 방지
- (3) full, empty 신호로 FSM 흐름 제어

4. UART 및 FIFO 연동

▶ UART TOP module



4. UART 및 FIFO 연동

▶ 주요 코드 : FIFO -> DEPTH 증가

- DEPTH를 30으로 증가
- 포인터 비트 수를 5비트로 증가



- FIFO에 저장할 수 있는 데이터 개수 증가

```
module fifo (  
    input        clk,  
    input        rst,  
    input        push,  
    input        pop,  
    input [7:0]  push_Data,  
    output       full,  
    output       empty,  
    output [7:0] pop_data  
);  
  
    wire [4:0] w_w_ptr, w_r_ptr;  
  
    register_file #(  
        .DEPTH(30)  
    ) U_Reg_File (  
        .clk (clk),  
        .wr_en(push & (~full)),  
        .wdata(push_Data),  
        .w_ptr(w_w_ptr),  
        .r_ptr(w_r_ptr),  
        .rdata(pop_data)  
    );
```

4. UART 및 FIFO 연동

▶ 주요 코드 : start 신호 분리

- start를 dist_done과 dht_done으로 분리



- FSM 상태 사이에 값이 사라지지 않도록
r_dht_done, r_dist_done에 저장

```
always @(*) begin
    n_state      = c_state;
    send_data_next = send_data_reg;
    send_next     = send_reg;
    send_cnt_next = send_cnt_reg;
    n_dist_done   = r_dist_done;
    n_dht_done    = r_dht_done;
    case (c_state)
        00: begin
            send_cnt_next = 0;
            if (dht_done | dist_done) begin
                n_state = 1;
                n_dist_done = dist_done;
                n_dht_done = dht_done;
            end
        end
    end
```

5. 동작 시연 ▶ SR sensor



5. 동작 시연 ▶ DHT11 sensor



6. 고찰

- (1) Top module을 작성 하기 위해서는 하위 module의 구성과 인스턴스화의 구성이 중요하다는 것을 깨달았습니다.
- (2) FIFO 구조를 연동시키면서 각 모듈이 독립적으로 작동하면서 통신 과정에서의 간섭 없이 동작하는 것을 보고 FIFO의 역할을 배웠습니다.
- (3) UART/FIFO 모듈을 타 모듈과 연결하며, 각자의 데이터폭, 클럭, 타이밍 동기를 맞추는 것이 중요하고 쉽지 않다는 것을 느꼈습니다.